

Spring Data JPA & Hibernate

JPA Hands-on 1

1. Need and Benefits of ORM

What is ORM?

ORM (Object Relational Mapping) is a technique that maps Java objects to database tables.

Example:

Java Class	Database Table
Employee	EMPLOYEE
Student	STUDENT

Instead of writing SQL manually, ORM converts Java objects into database records.

Benefits of ORM

- * Reduces SQL coding
- * Database independent
- * Improves productivity
- * Handles transactions
- * Easier maintenance
- * Supports object-oriented programming

Advantages

- * Faster development
- * Reusable code
- * Automatic table mapping
- * Less boilerplate code

Disadvantages

- * Learning curve
- * Slight performance overhead
- * Complex queries may still require SQL

2. Need and Benefits of Spring Data JPA

Evolution

JDBC



Hibernate XML Configuration



Hibernate Annotation Configuration



Spring Data JPA

Why Spring Data JPA?

Spring Data JPA reduces Hibernate boilerplate code.

Without Spring Data JPA:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
session.save(employee);
tx.commit();
session.close();
```

With Spring Data JPA:

```
repository.save(employee);
```

Benefits

- * Automatic CRUD operations
- * Query generation
- * Pagination support
- * Sorting support
- * Integration with Spring Boot
- * Less code

3. Core Objects of Hibernate Framework

SessionFactory

- * Creates Session objects
- * Heavyweight object
- * Created once per application

```
SessionFactory factory;
```

Session

- * Used to interact with database

```
Session session =
factory.openSession();
```

Transaction

- * Maintains data consistency

```
Transaction tx =
session.beginTransaction();
```

Connection Provider

- * Provides database connections

Configuration

- * Reads Hibernate configuration file

```
Configuration cfg =
new Configuration().configure();
```

4. ORM Implementation using Hibernate

XML Configuration

Employee.java

```
public class Employee {  
    private int id;  
    private String name;  
}
```

Employee.hbm.xml

```
<class name="Employee" table="EMPLOYEE">  
    <id name="id"/>  
    <property name="name"/>  
</class>
```

hibernate.cfg.xml

```
<hibernate-configuration>  
    <session-factory>  
    </session-factory>  
</hibernate-configuration>
```

Annotation Configuration :

Employee.java

```
@Entity  
@Table(name="EMPLOYEE")  
public class Employee {  
    @Id  
    private int id;  
    @Column  
    private String name;  
}
```

Common Annotations :

Annotation	Purpose
@Entity	Marks entity class
@Table	Maps table
@Id	Primary key
@Column	Maps column

Hibernate Workflow :

```
Configuration cfg =  
new Configuration().configure();  
SessionFactory factory =  
cfg.buildSessionFactory();  
Session session =  
factory.openSession();  
Transaction tx =  
session.beginTransaction();  
session.save(emp);  
tx.commit();
```

```
session.close();
```

5. Difference between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	Implementation	Abstraction
Developed By	Java Community	Hibernate Team	Spring
CRUD Support	No	Yes	Yes
Boilerplate Code	High	Medium	Very Low
Query Support	Limited	Strong	Strong

Relationship

Spring Data JPA



Hibernate



JPA

Viva Question

What is JPA?

JPA (Java Persistence API) is a specification for managing relational data in Java applications.

Is Hibernate JPA?

No. Hibernate is an implementation of JPA.

6. DML Operations using Spring Data JPA

Entity

```
@Entity
public class Book {
    @Id
    private Long id;
    private String title;
}
```

Repository :

```
import org.springframework.data.jpa.repository.JpaRepository;
public interface BookRepository
extends JpaRepository<Book, Long> {
}
```

Insert

```
repository.save(book);
```

Update

```
repository.save(book);
```

Find By ID

```
repository.findById(1L);
```

Delete

```
repository.deleteById(1L);
```

Query Method

```
List<Book> findByTitle(String title);
```

Usage:

```
repository.findByTitle("Spring");
```

application.properties :

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

ddl-auto Values

Value	Purpose
create	Creates table every run
update	Updates schema
validate	Validates schema
none	No action

Hands-On 1: Spring Data JPA

1. application.properties :

Spring Logs

```
logging.level.org.springframework=info  
logging.level.com.cognizant=debug
```

Hibernate Logs

```
logging.level.org.hibernate.SQL=trace  
logging.level.org.hibernate.type.descriptor.sql=trace
```

Log Pattern

```
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p  
%-25.25logger{25} %25M %4L %m%n
```

Database Configuration

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn  
spring.datasource.username=root  
spring.datasource.password=root
```

Hibernate Configuration

```
spring.jpa.hibernate.ddl-auto=validate  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

2. Country Table :

```
create table country(  
    code varchar(2) primary key,  
    name varchar(50)  
);
```

```
insert into country values ('IN','India');  
insert into country values ('US','United States of America');
```

3. Country.java :

```
package com.cognizant.ormlearn.model;  
  
import jakarta.persistence.Entity;  
import jakarta.persistence.Id;  
import jakarta.persistence.Table;  
import jakarta.persistence.Column;  
  
@Entity  
@Table(name="country")  
public class Country {  
  
    @Id  
    @Column(name="code")  
    private String code;  
  
    @Column(name="name")  
    private String name;  
  
    public String getCode() {  
        return code;  
    }  
  
    public void setCode(String code) {  
        this.code = code;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public String toString() {  
        return "Country [code=" + code + ", name=" + name + "];"  
    }  
}
```

4. CountryRepository.java :

```
package com.cognizant.ormlearn.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
import com.cognizant.ormlearn.model.Country;
```

```

@Repository
public interface CountryRepository
    extends JpaRepository<Country, String> {

}

```

5. CountryService.java :

```

package com.cognizant.ormlearn.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}

```

6. OrmLearnApplication.java :

```

package com.cognizant.ormlearn;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);
    }
}

```

```

    LOGGER.info("Inside main");

    countryService = context.getBean(CountryService.class);

    testGetAllCountries();
}

private static void testGetAllCountries() {

    LOGGER.info("Start");

    List<Country> countries =
        countryService.getAllCountries();

    LOGGER.debug("countries={}", countries);

    LOGGER.info("End");
}
}

```

Output :

```

Inside main
Start
countries=[Country [code=IN, name=India],
Country [code=US, name=United States of America]]
End

```

Hands-On 2: Hibernate XML Configuration Implementation

Employee.java :

```

public class Employee {

    private int id;
    private String firstName;
    private String lastName;
    private int salary;

    // getters and setters
}

```

Employee.hbm.xml :

```

<hibernate-mapping>

    <class name="Employee" table="EMPLOYEE">

        <id name="id" column="id">
            <generator class="native"/>
        </id>

        <property name="firstName" column="first_name"/>
        <property name="lastName" column="last_name"/>
        <property name="salary" column="salary"/>

    </class>

```


</hibernate-mapping>

2. hibernate.cfg.xml :

<hibernate-configuration>

<session-factory>

<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/test
</property>

<property name="hibernate.connection.username">
root
</property>

<property name="hibernate.connection.password">
root
</property>

<mapping resource="Employee.hbm.xml"/>

</session-factory>

</hibernate-configuration>

3. SessionFactory

```
Configuration cfg =  
    new Configuration().configure();
```

```
SessionFactory factory =  
    cfg.buildSessionFactory();
```

4. Session

```
Session session =  
    factory.openSession();
```

5. Transaction

```
Transaction tx =  
    session.beginTransaction();
```

6. beginTransaction()

```
Transaction tx =  
    session.beginTransaction();
```

7. commit()

```
tx.commit();
```

8. rollback()

```
tx.rollback();
```

9. session.save()

```
Employee emp = new Employee();
```

```
emp.setFirstName("John");  
emp.setLastName("David");  
emp.setSalary(50000);
```

```
session.save(emp);
```

SQL Generated :

```
insert into EMPLOYEE values (...);
```

10. session.createQuery().list()

```
List<Employee> employees =  
    session.createQuery("FROM Employee")  
        .list();
```

SQL Generated :

```
select * from EMPLOYEE;
```

Returns :

```
List<Employee>
```

11. session.get() :

```
Employee emp =  
    session.get(Employee.class, 1);
```

SQL Generated :

```
select * from EMPLOYEE where id=1;
```

Returns :

Employee object

12. session.delete() :

```
Employee emp =  
    session.get(Employee.class, 1);
```

```
session.delete(emp);
```

SQL Generated :

```
delete from EMPLOYEE where id=1;
```

Hands-On 3: Hibernate Annotation Configuration Walkthrough

1. Employee Persistence Class :

```

import javax.persistence.*;

@Entity
@Table(name = "EMPLOYEE")
public class Employee {

    @Id
    @GeneratedValue
    @Column(name = "id")
    private int id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "salary")
    private int salary;

    // Getters and Setters
}

```

2. Object to Relational Mapping

Hibernate maps Java class fields to database columns using annotations.

3. @Entity :

```

@Entity
public class Employee {
}

```

4. @Table

```

@Table(name = "EMPLOYEE")

```

5. @Id

```

@Id
private int id;

```

6. @GeneratedValue

```

@Id
@GeneratedValue
private int id;

```

7. @Column

```

@Column(name="first_name")
private String firstName;

```

8. Hibernate Configuration (hibernate.cfg.xml)

```

<hibernate-configuration>

    <session-factory>

```

```
<property name="hibernate.connection.driver_class">
    com.mysql.cj.jdbc.Driver
</property>
```

```
<property name="hibernate.connection.url">
    jdbc:mysql://localhost:3306/test
</property>
```

```
<property name="hibernate.connection.username">
    root
</property>
```

```
<property name="hibernate.connection.password">
    root
</property>
```

```
<property name="hibernate.dialect">
    org.hibernate.dialect.MySQL8Dialect
</property>
```

```
<mapping class="Employee"/>
```

```
</session-factory>
```

```
</hibernate-configuration>
```

9. Dialect

```
<property name="hibernate.dialect">
    org.hibernate.dialect.MySQL8Dialect
</property>
```

10. Driver

```
<property name="hibernate.connection.driver_class">
    com.mysql.cj.jdbc.Driver
</property>
```

11. Connection URL

```
<property name="hibernate.connection.url">
    jdbc:mysql://localhost:3306/test
</property>
```

12. Username

```
<property name="hibernate.connection.username">
    root
</property>
```

13. Password

```
<property name="hibernate.connection.password">
    root
</property>
```

Hibernate Workflow

```
Configuration cfg =
    new Configuration().configure();
```

```
SessionFactory factory =  
    cfg.buildSessionFactory();  
  
Session session =  
    factory.openSession();  
  
Transaction tx =  
    session.beginTransaction();  
  
session.save(employee);  
  
tx.commit();  
  
session.close();
```

Hands-On 4: Difference Between JPA, Hibernate and Spring Data JPA

1. Java Persistence API (JPA)

Definition

- * JPA (Java Persistence API) is a Specification (JSR 338) for persisting Java objects into databases.
- * It defines interfaces and annotations.
- * JPA itself does not provide implementation.

Example Annotations

```
@Entity  
@Table(name="employee")  
public class Employee {  
    @Id  
    private int id;  
    private String name;  
}
```

2. Hibernate

Definition

- * Hibernate is an ORM Tool.
- * Hibernate is one of the most popular implementations of JPA.
- * It converts Java objects into database records automatically.

Hibernate Example

```
Session session = factory.openSession();  
Transaction tx =  
    session.beginTransaction();  
session.save(employee);  
tx.commit();  
session.close();
```

Responsibilities

- * Object Relational Mapping

- * SQL Generation
- * Session Management
- * Transaction Management

3. Spring Data JPA

Definition

- * Spring Data JPA is not a JPA implementation.
- * It is an abstraction layer over Hibernate/JPA.
- * It reduces boilerplate code.

Spring Data JPA Example

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {
}
@Autowired
private EmployeeRepository repository;
@Transactional
public void addEmployee(Employee employee) {
    repository.save(employee);
}
```

Architecture

```
Application
  ↓
Spring Data JPA
  ↓
Hibernate
  ↓
JPA Specification
  ↓
Database
```

Hibernate vs Spring Data JPA

Hibernate Code

```
Session session = factory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();
    session.save(employee);
    tx.commit();
} catch (Exception e) {
    tx.rollback();
} finally {
    session.close();
}
```

Spring Data JPA Code

```
@Autowired
private EmployeeRepository repository;
```

```
@Transactional
public void addEmployee(Employee employee) {
    repository.save(employee);
}
```

Observation

- * Hibernate requires manual session handling.
- * Spring Data JPA automatically manages transactions and repositories.
- * Much less code.

Advantages of Spring Data JPA

- * Less boilerplate code
- * Automatic CRUD methods
- * Built-in transaction management
- * Query method support
- * Pagination and Sorting
- * Easy Spring Boot integration

Example:

```
repository.findAll();
repository.findById(1);
repository.save(employee);
repository.deleteById(1);
```

Hands-On 5: Implement Services for Managing Country

1. Hibernate ddl-auto Configuration

```
spring.jpa.hibernate.ddl-auto=validate
```

2. Country Entity

```
package com.cognizant.ormlearn.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "country")
```

```
public class Country {
```

```
    @Id
```

```
    @Column(name = "co_code")
```

```
    private String code;
```

```
    @Column(name = "co_name")
```

```
    private String name;
```

```
    public Country() {
```

```
    }
```

```
    public Country(String code, String name) {
```

```
        this.code = code;
```

```
        this.name = name;
```

```
    }
```

```

public String getCode() {
    return code;
}

public void setCode(String code) {
    this.code = code;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

@Override
public String toString() {
    return "Country [code=" + code + ", name=" + name + "]";
}
}

```

3. CountryRepository

```

package com.cognizant.ormlearn.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

    List<Country> findByNameContainingIgnoreCase(String name);
}

```

4. CountryService

```

package com.cognizant.ormlearn.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional

```



```

public List<Country> getAllCountries() {
    return countryRepository.findAll();
}

@Transactional
public Country findCountryByCode(String code) {
    return countryRepository.findById(code).orElse(null);
}

@Transactional
public void addCountry(Country country) {
    countryRepository.save(country);
}

@Transactional
public void updateCountry(Country country) {
    countryRepository.save(country);
}

@Transactional
public void deleteCountry(String code) {
    countryRepository.deleteById(code);
}

@Transactional
public List<Country> searchCountries(String name) {
    return countryRepository.findByNameContainingIgnoreCase(name);
}
}

```

5. Testing in OrmLearnApplication

```

package com.cognizant.ormlearn;

import java.util.List;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static CountryService countryService;

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);

        countryService =
            context.getBean(CountryService.class);

        testFindCountry();
        testAddCountry();
        testUpdateCountry();
        testDeleteCountry();
    }
}

```

```

    testSearchCountry();
}

private static void testFindCountry() {
    System.out.println(
        countryService.findCountryByCode("IN"));
}

private static void testAddCountry() {
    Country country =
        new Country("XX", "Test Country");

    countryService.addCountry(country);

    System.out.println("Country Added");
}

private static void testUpdateCountry() {
    Country country =
        new Country("XX", "Updated Country");

    countryService.updateCountry(country);

    System.out.println("Country Updated");
}

private static void testDeleteCountry() {
    countryService.deleteCountry("XX");

    System.out.println("Country Deleted");
}

private static void testSearchCountry() {
    List<Country> countries =
        countryService.searchCountries("Uni");

    countries.forEach(System.out::println);
}
}

```

Output :

Country [code=IN, name=India]

Country Added

Country Updated

Country Deleted

Country [code=GB, name=United Kingdom]

Country [code=US, name=United States]

Country [code=UM, name=United States Minor Outlying Islands]

Hands-On 6: Find a Country Based on Country Code

1. Create Custom Exception :

```
package com.cognizant.ormlearn.service.exception;

public class CountryNotFoundException extends Exception {

    public CountryNotFoundException(String message) {
        super(message);
    }
}
```

2. Update CountryService

```
package com.cognizant.ormlearn.service;

import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public Country findCountryByCode(String countryCode)
        throws CountryNotFoundException {

        Optional<Country> result =
            countryRepository.findById(countryCode);

        if (!result.isPresent()) {
            throw new CountryNotFoundException(
                "Country Not Found");
        }

        Country country = result.get();

        return country;
    }
}
```

3. Update OrmLearnApplication

```
package com.cognizant.ormlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
```

```
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;
```

```
@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    private static CountryService countryService;

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(
                OrmLearnApplication.class, args);

        countryService =
            context.getBean(CountryService.class);

        getCountryTest();
    }

    private static void getCountryTest() {

        LOGGER.info("Start");

        try {

            Country country =
                countryService.findCountryByCode("IN");

            LOGGER.debug("Country: {}", country);

        } catch (CountryNotFoundException e) {

            LOGGER.error(e.getMessage());

        }

        LOGGER.info("End");
    }
}
```

Output :

Start

Country: Country [code=IN, name=India]

End

Hands-On 7: Add a New Country

1. Add Method in CountryService :

CountryService.java

```

@Transactional
public void addCountry(Country country) {

    countryRepository.save(country);
}

```

Complete Method :

```

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public void addCountry(Country country) {

        countryRepository.save(country);
    }
}

```

2. Add Test Method in OrmLearnApplication :

```

private static void testAddCountry() {

    LOGGER.info("Start");

    Country country = new Country();

    country.setCode("NP");
    country.setName("Nepal");

    countryService.addCountry(country);

    try {

        Country addedCountry =
            countryService.findCountryByCode("NP");

        LOGGER.debug("Added Country: {}", addedCountry);

    } catch (CountryNotFoundException e) {

        LOGGER.error(e.getMessage());
    }

    LOGGER.info("End");
}

```

3. Invoke Test Method in main()

```

public static void main(String[] args) {

    ApplicationContext context =
        SpringApplication.run(
            OrmLearnApplication.class, args);

    countryService =
        context.getBean(CountryService.class);
}

```

```
    testAddCountry();  
}
```

Output :

INFO Start

DEBUG Added Country: Country [code=NP, name=Nepal]

INFO End

Hands-On 8: Update a Country Based on Code

1. Add updateCountry() Method in CountryService

```
@Transactional  
public void updateCountry(String code, String name)  
    throws CountryNotFoundException {  
  
    Optional<Country> result =  
        countryRepository.findById(code);  
  
    if (!result.isPresent()) {  
        throw new CountryNotFoundException(  
            "Country Not Found");  
    }  
  
    Country country = result.get();  
  
    country.setName(name);  
  
    countryRepository.save(country);  
}
```

Complete Service Method

```
@Service  
public class CountryService {  
  
    @Autowired  
    private CountryRepository countryRepository;  
  
    @Transactional  
    public void updateCountry(String code, String name)  
        throws CountryNotFoundException {  
  
        Optional<Country> result =  
            countryRepository.findById(code);  
  
        if (!result.isPresent()) {  
            throw new CountryNotFoundException(  
                "Country Not Found");  
        }  
  
        Country country = result.get();  
  
        country.setName(name);  
    }  
}
```

```

        countryRepository.save(country);
    }
}

```

2. Add Test Method in OrmLearnApplication

```

private static void testUpdateCountry() {
    LOGGER.info("Start");

    try {
        countryService.updateCountry(
            "IN",
            "Republic of India");

        Country country =
            countryService.findCountryByCode("IN");

        LOGGER.debug("Updated Country: {}", country);
    } catch (CountryNotFoundException e) {
        LOGGER.error(e.getMessage());
    }

    LOGGER.info("End");
}

```

3. Invoke Test Method in main()

```

public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(
            OrmLearnApplication.class, args);

    countryService =
        context.getBean(CountryService.class);

    testUpdateCountry();
}

```

Output :

INFO Start

DEBUG Updated Country:
Country [code=IN, name=Republic of India]

INFO End

Hands-On 9: Delete a Country Based :

1. Add deleteCountry() Method in CountryService

@Transactional

```
public void deleteCountry(String countryCode) {  
    countryRepository.deleteByld(countryCode);  
}
```

Complete Method

```
@Service  
public class CountryService {  
  
    @Autowired  
    private CountryRepository countryRepository;  
  
    @Transactional  
    public void deleteCountry(String countryCode) {  
        countryRepository.deleteByld(countryCode);  
    }  
}
```

2. Add Test Method in OrmLearnApplication

```
private static void testDeleteCountry() {  
    LOGGER.info("Start");  
    countryService.deleteCountry("NP");  
    LOGGER.debug("Country with code NP deleted");  
    LOGGER.info("End");  
}
```

3. Invoke Test Method in main()

```
public static void main(String[] args) {  
    ApplicationContext context =  
        SpringApplication.run(  
            OrmLearnApplication.class, args);  
  
    countryService =  
        context.getBean(CountryService.class);  
  
    testDeleteCountry();  
}
```

Output

INFO Start

DEBUG Country with code NP deleted

INFO End
